

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Организация процессов и программирование в среде
Linux»
Тема: Создание и идентификация процессов

Студент гр. 3311

Баймухамедов Р. Р.

Преподаватель

Лазарев Э. А.

Санкт-Петербург

2026

ЦЕЛЬ РАБОТЫ

Изучить и использовать системные функции, обеспечивающие порождение и идентификацию процессов.

ЗАДАНИЕ

1. Разработать программу, которая порождает два процесса-потомка. Первый потомок должен быть создан функцией `fork`, а второй - функцией `vfork` с последующей заменой выполняемой программы. Предок и оба потомка должны вывести в один файл свои атрибуты с указанием имени процесса. Имя выходного файла и задержки трёх процессов задаются при запуске программы.
2. Откомпилировать программу и запустить её три раза с различными сочетаниями задержек.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

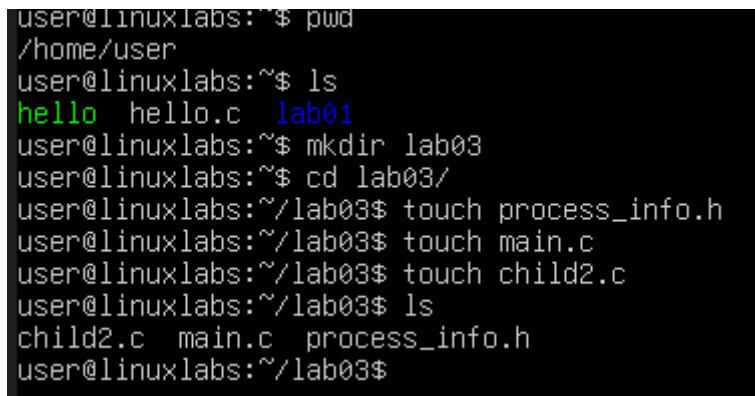
Процесс в Linux имеет уникальный идентификатор PID и связан с породившим его процессом через PPID. К атрибутам процесса также относятся идентификатор сессии SID, идентификатор группы процессов PGID, реальные и эффективные идентификаторы пользователя UID и EUID, а также группы GID и EGID.

Функция `fork` создаёт копию текущего процесса. Функция `vfork` временно разделяет память с предком и блокирует его до вызова `exec` или `_exit`. Семейство функций `exec` заменяет выполняемую программу процесса другой программой. В разработанной программе функция `sleep` задаёт задержку перед записью атрибутов, поэтому порядок блоков в выходном файле определяется значениями задержек.

2 РАЗРАБОТКА ПРОГРАММЫ

2.1 Подготовка рабочего каталога

Программа состоит из главного файла main.c, отдельной программы child2.c для второго потомка и общего заголовочного файла process_info.h.



```
user@linuxlabs:~$ pwd
/home/user
user@linuxlabs:~$ ls
hello hello.c lab01
user@linuxlabs:~$ mkdir lab03
user@linuxlabs:~$ cd lab03/
user@linuxlabs:~/lab03$ touch process_info.h
user@linuxlabs:~/lab03$ touch main.c
user@linuxlabs:~/lab03$ touch child2.c
user@linuxlabs:~/lab03$ ls
child2.c main.c process_info.h
user@linuxlabs:~/lab03$
```

Рисунок 1 - Подготовка рабочего каталога и файлов программы

2.2 Заголовочный файл process_info.h

В заголовочном файле реализованы проверка задержки и общая функция записи атрибутов процесса.

```
#ifndef PROCESS_INFO_H
#define PROCESS_INFO_H

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

// transform string argument at delay value
static unsigned parse_delay(const char *value){
    char *end = NULL;
    unsigned long delay = strtoul(value, &end, 10);

    if (*value == '\\0' || *end != '\\0' || delay > 3600) {
        fprintf(stderr, "Incorrect delay: %s\\n", value);
        exit(EXIT_FAILURE);
    }

    return (unsigned)delay;
}

// process delay and write its attributes in shared file
static void write_attributes(const char *process_name,
```

```

const char *output_file, unsigned delay){
    sleep(delay);

    int fd = open(output_file, O_WRONLY | O_APPEND);
    if (fd == -1) {
        perror("open");
        _exit(EXIT_FAILURE);
    }

    dprintf(fd,
        "%s:\n"
        "delay = %u c\n"
        "PID = %ld\n"
        "PPID = %ld\n"
        "SID = %ld\n"
        "PGID = %ld\n"
        "UID = %ld\n"
        "EUID = %ld\n"
        "GID = %ld\n"
        "EGID = %ld\n\n",
        process_name,
        delay,
        (long)getpid(),
        (long)getppid(),
        (long)getsid(0),
        (long)getpgid(0),
        (long)getuid(),
        (long)geteuid(),
        (long)getgid(),
        (long)getegid());

    close(fd);
}

#endif

```

2.3 Главная программа main.c

Главная программа создаёт первого потомка с помощью `fork`, второго - с помощью `vfork`, а затем заменяет программу второго потомка вызовом `execl`.

```

#define _GNU_SOURCE

#include "process_info.h"
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    // name of file and three delay values

```

```

if (argc != 5) {
    fprintf(stderr,
        "Use: %s FILE PARENT_DELAY "
        "CHILD1_DELAY CHILD2_DELAY\n",
        argv[0]);
    return EXIT_FAILURE;
}

const char *output_file = argv[1];
unsigned parent_delay = parse_delay(argv[2]);
unsigned child1_delay = parse_delay(argv[3]);
unsigned child2_delay = parse_delay(argv[4]);

// create new file and write launch parameters
int fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC,
0644);
if (fd == -1) {
    perror("open");
    return EXIT_FAILURE;
}

dprintf(fd,
    "parameters:\n"
    "output file = %s\n"
    "parent delay = %u c\n"
    "child 1 delay = %u c\n"
    "child 2 delay = %u c\n\n",
    output_file,
    parent_delay,
    child1_delay,
    child2_delay);
close(fd);

// first child continue execution current program
pid_t child1 = fork();
if (child1 == -1) {
    perror("fork");
    return EXIT_FAILURE;
}
if (child1 == 0) {
    write_attributes("Child 1 (fork)", output_file,
child1_delay);
    _exit(EXIT_SUCCESS);
}

// second child immediately change program via execl
pid_t child2 = vfork();
if (child2 == -1) {
    perror("vfork");
    return EXIT_FAILURE;
}

```

```

        if (child2 == 0) {
            execl("./child2", "child2", output_file, argv[4],
(char *)NULL);
            _exit(127);
        }

        write_attributes("Parent", output_file, parent_delay);

        // parent wait ending of both childs
        waitpid(child1, NULL, 0);
        waitpid(child2, NULL, 0);

        return EXIT_SUCCESS;
}

```

2.4 Программа второго потомка child2.c

После вызова `execl` второй потомок выполняет отдельную программу `child2`, которая записывает свои атрибуты в общий файл.

```

#define _GNU_SOURCE

#include "process_info.h"

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "Use %s FILE DELAY\n", argv[0]);
        return EXIT_FAILURE;
    }

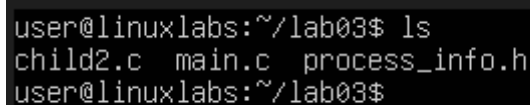
    unsigned delay = parse_delay(argv[2]);
    // this programm execute as second child after execl
    write_attributes("Child 2 (vfork + exec)", argv[1],
delay);

    return EXIT_SUCCESS;
}

```

3 КОМПИЛЯЦИЯ ПРОГРАММЫ

Сначала компилируется программа второго потомка, затем главная программа.



```

user@linuxlabs:~/lab03$ ls
child2.c main.c process_info.h
user@linuxlabs:~/lab03$

```

```

gcc child2.c -o child2
gcc main.c -o process_lab3

```

```
user@linuxlabs:~/lab03$ ls -la
total 60
drwxrwxr-x 2 user user 4096 Sep 17 20:17 .
drwxr-x--- 8 user user 4096 Sep 15 18:05 ..
-rwxrwxr-x 1 user user 16808 Sep 17 20:14 child2
-rw-rw-r-- 1 user user 352 Sep 17 20:06 child2.c
-rw-rw-r-- 1 user user 1506 Sep 17 20:17 main.c
-rw-rw-r-- 1 user user 1116 Sep 17 20:10 process_info.h
-rwxrwxr-x 1 user user 16952 Sep 17 20:17 process_lab3
user@linuxlabs:~/lab03$
```

Рисунок 2 - Компиляция программы

4 ЗАПУСК ПРОГРАММЫ

4.1 Первый запуск

В первом запуске наименьшая задержка задана первому потомку, затем второму потомку и предку.

```
user@linuxlabs:~/lab03$ ./process_lab3 result1.txt 3 1 2
user@linuxlabs:~/lab03$ cat result1.txt
parameters:
output file = result1.txt
parent delay = 3 c
child 1 delay = 1 c
child 2 delay = 2 c

Child 1 (fork):
delay = 1
PID = 2795
PPID = 2794
SID = 1183
PGID = 2794
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Child 2 (vfork + exec):
delay = 2
PID = 2796
PPID = 2794
SID = 1183
PGID = 2794
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Parent:
delay = 3
PID = 2794
PPID = 1183
SID = 1183
PGID = 2794
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000
```

Рисунок 3 - Результат первого запуска программы

4.2 Второй запуск

Во втором запуске наименьшая задержка задана предку, затем второму и первому потомкам.

```
user@linuxlabs:~/lab03$ ./process_lab3 result2.txt 1 3 2
user@linuxlabs:~/lab03$ cat result2.txt
parameters:
output file = result2.txt
parent delay = 1 c
child 1 delay = 3 c
child 2 delay = 2 c

Parent:
delay = 1
PID = 2830
PPID = 1183
SID = 1183
PGID = 2830
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Child 2 (vfork + exec):
delay = 2
PID = 2832
PPID = 2830
SID = 1183
PGID = 2830
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Child 1 (fork):
delay = 3
PID = 2831
PPID = 2830
SID = 1183
PGID = 2830
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000
```

Рисунок 4 - Результат второго запуска программы

4.3 Третий запуск

В третьем запуске наименьшая задержка задана первому потомку, затем предку и второму потомку.

```
user@linuxlabs:~/lab03$ ./process_lab3 result3.txt 2 1 3
user@linuxlabs:~/lab03$ cat result3.txt
parameters:
output file = result3.txt
parent delay = 2 c
child 1 delay = 1 c
child 2 delay = 3 c

Child 1 (fork):
delay = 1
PID = 2863
PPID = 2862
SID = 1183
PGID = 2862
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Parent:
delay = 2
PID = 2862
PPID = 1183
SID = 1183
PGID = 2862
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000

Child 2 (vfork + exec):
delay = 3
PID = 2864
PPID = 2862
SID = 1183
PGID = 2862
UID = 1000
EUID = 1000
GID = 1000
EGID = 1000
```

Рисунок 5 - Результат третьего запуска программы

5 АНАЛИЗ РЕЗУЛЬТАТОВ

Во всех трёх запусках процессы получили различные PID. Для потомков значение PPID соответствует PID процесса-предка. Процессы принадлежат одной сессии и наследуют группу процессов, а реальные и эффективные идентификаторы пользователя и группы совпадают, поскольку права доступа в программе не изменялись.

Порядок записи блоков изменялся в соответствии с задержками: в первом запуске данные записали потомок 1, потомок 2 и предок; во втором - предок, потомок 2 и потомок 1; в третьем - потомок 1, предок и потомок 2. Таким образом, задержки позволяют управлять наблюдаемым порядком вывода процессов.

ВЫВОД

В ходе лабораторной работы были изучены функции получения атрибутов процессов, а также функции `fork`, `vfork` и `exec`. Разработана и проверена программа, создающая двух потомков и записывающая атрибуты трёх процессов в общий файл. Три запуска с разными задержками показали, что порядок вывода определяется моментом возобновления выполнения каждого процесса.